FATEC – FACULDADE DE TECNOLOGIA DE RIBEIRÃO PRETO

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

LEONARDO DE CASTRO

ESTRUTURA DE DADOS

PROJETO PRÁTICO INDIVIDUAL – ÁRVORE BINÁRIA DE BUSCA (BST) E ESTRUTURA LINEAR

RIBEIRÃO PRETO 2026

Sumário

1.Tema Escolhido e Modelagem.....3
2. Funcionamento da BST.....3
3. Funcionamento da Estrutura Linear (Fila).....4
4. Exemplos de Testes Realizados.....4
5. Breve Análise de Complexidade.....5
6. Conclusão e Limitações.....5
7. Declaração de uso de IA.....5

1.Tema Escolhido e Modelagem

O tema escolhido para este projeto foi o C – Biblioteca, cujo objetivo é simular o cadastro de livros. Para modelar isso na linguagem C, foi utilizada uma estrutura ('struct') principal chamada 'Livro' que guarda os seguintes campos exigidos:

- ID: Um número inteiro positivo, que funciona como código único para cada livro.
- Título e Autor**:** Textos com até 79 caracteres.
- Ano de publicação: Um número inteiro que passa por uma validação para aceitar apenas os anos entre 1 e 2026.

Para a Árvore Binária de Busca (BST), utaliza-se um nó que guarda esse Livro inteiro, junto com dois ponteiros (um para o lado esquerdo e outro para o direito).

Já para a estrutura linear, optou-se por construir uma Fila. A modelagem da fila foi mantida simples, conforme solicitado: cada nó guarda apenas um número inteiro (que simula o ID) e um ponteiro para o próximo da fila. Para facilitar, a estrutura da fila possui dois ponteiros de controle, um marcando o início e outro o fim.

2. Funcionamento da BST

A árvore é o coração do cadastro. Ela organiza os livros automaticamente baseada no ID, seguindo a regra de ouro das BSTs: o que é menor que o nó atual vai para a esquerda, e o que é maior vai para a direita.

- **Inserção:** O sistema lê o ID fornecido. Se ele for menor que o ID da raiz, o programa desce para a esquerda; se maior, para a direita. O sistema está programado para não aceitar IDs duplicados e também bloqueia números negativos e anos inválidos logo no início do menu.
- **Busca:** Segue a mesma lógica de navegação da inserção. Se encontrar o ID, ele imprime as informações do livro. Se bater em um “fundo falso” (ponteiro vazio), o programa avisa que o livro não existe.
- **Remoção:** É a parte mais complexa da árvore. Se o livro que vou apagar for uma “folha” (não tem filhos), ele só é apagado. Se tiver um filho, o filho toma o lugar dele. Mas se tiver dois filhos, o programa procura o nó com o menor valor do lado direito, copia esse valor para o nó que será “apagado” e depois remove a duplicata lá embaixo, garantindo que a árvore não desmorone.

- **Listagem Em-Ordem:** O código visita tudo à esquerda, depois o nó atual, e depois a direita. Isso resulta em uma lista de livros impressa sempre em ordem crescente, do menor ID para o maior.

3. Funcionamento da Estrutura Linear (Fila)

A Fila opera de forma totalmente independente da árvore de livros. Ela serve apenas para armazenar IDs utilizando o modelo FIFO (*First In, First Out* – o primeiro a entrar é o primeiro a sair).

Inserção (Enqueue): Como a estrutura tem um ponteiro que acompanha sempre quem é o último da fila (fim), é possível colocar um novo ID no final imediatamente, sem precisar varrer todos os elementos. Como o projeto exige, a fila aceita IDs repetidos e não valida se o ID pertence a algum livro cadastrado na árvore.

Remoção (Dequeue): O item que está no início da fila é apagado e o ponteiro de início anda uma casa para a frente. Se o usuário tentar remover alguém em uma fila vazia, o sistema apenas imprime um aviso, sem gerar erro e sem fechar o programa de forma brusca.

Exibição: O programa começa do ponteiro inicial e vai imprimindo caixinha por caixinha até alcançar o final da fila.

4. Exemplos de Testes Realizados

Para garantir que o código estava coeso, foram realizados alguns testes principais:

- **Teste de Limites (Domínio):** Tentativa de inserir um livro com o ano 2030 e um ID -10. O sistema travou a operação no mesmo instante exibindo erros de "Ano invalido" e "ID deve ser positivo", provando que as validações do menu funcionam.
- **Teste de Remoção Crítica:** Foram inseridos na árvore os IDs 50, 30, 70, 60 e 80. Ao ser solicitado para remover o 70 (que tinha dois filhos). A operação foi um sucesso e, ao listar em-ordem, a tela exibiu perfeitamente 30, 50, 60, 80, provando que os ponteiros foram religados corretamente.
- **Teste de Sobrevivência (Fila Vazia):** Ao selecionar a a opção de remover elemento com a fila recém-inicializada (zerada). O terminal apenas retornou "Aviso:

A estrutura linear esta vazia" e o menu continuou rodando normalmente, não vazando memória nem travando a aplicação.

5. Breve Análise de Complexidade

BST (Inserir, Buscar e Remover): Em uma situação normal, essas operações têm complexidade O(log n), o que é muito bom e rápido, pois a árvore vai “cortando os caminhos pela metade”. Contudo, o pior caso é O(n). Se um usuário cadastrar os livros já ordenados (ex: IDs 1, 2, 3, 4, 5), a árvore perde seu formato e vira uma “linha reta”, obrigando o programa a passar por todos os nós para achar o último.

Fila Dinâmica: As operações de inserir e remover têm complexidade O(1) (constante). Como são utilizados ponteiros de início e fim, o computador não precisa pensar nem percorrer a fila; ele vai direto ao ponto. Apenas a função de exibir a fila possui complexidade O(n), já que, inevitavelmente, é necessário visitar todos os elementos para imprimi-los na tela.

6. Conclusão e Limitações

O desenvolvimento desse projeto serviu perfeitamente para colocar em prática os conceitos vistos em sala, pois utilizar de forma prática e ser confrontado diretamente com os empecilhos envolvendo esses conceitos e suas complexidades auxilia no entendimento não só da teoria como da prática (e dos erros que podemos enfrentar no caminho).

Como limitação principal do sistema, vale ressaltar o problema citado na análise de complexidade: a árvore não se auto-balanceia. Em uma biblioteca real com muitos volumes, se os livros fossem cadastrados em lote, a performance cairia drasticamente.

7. Declaração de uso de IA

Conforme diretrizes do edital, declaro que utilizei a inteligência artificial Gemini como tutor particular. O uso foi focado no suporte para estruturar a organização do versionamento em branches, para scripts de desenvolvimento de testes, como parceiro de raciocínio lógico para debugar as operações de ponteiros na remoção da árvore e para auxílio na

refatoração/adequação do código, além de trabalhar na revisão ortográfica do relatório e arquivo README.